

EJERCICIO PRÁCTICO: SISTEMA DE GESTIÓN DE INVENTARIO Y VENTAS

Construcción de una API REST con 3 Modelos Relacionados y Seguridad JWT

BACKEND | ES MODULES | 3 MODELOS (ORM) | POSTGRESQL | POSTMAN | JWT

Objetivo General

Diseñar e implementar una API REST modular para el control de inventario y ventas de un comercio tecnológico, incorporando 3 entidades/modelos principales (Categoria, Producto y Venta) con sus respectivas relaciones (1:N), un módulo de autenticación mediante JSON Web Tokens (JWT), middlewares de validación, controladores CRUD y un plan integral de pruebas en Postman.

Estructura General del Proyecto

Organiza tu código en carpetas de forma modular aplicando el patrón MVC por capas:

```
api_inventario-ventas/
├── src/
│   ├── config/
│   │   └── db.js          # Conexión a PostgreSQL mediante Sequelize
│   ├── models/
│   │   ├── Categoria.js   # Modelo 1: Categorías de productos
│   │   ├── Producto.js    # Modelo 2: Productos del inventario
│   │   ├── Venta.js       # Modelo 3: Registro de ventas realizadas
│   │   └── index.js       # Definición de relaciones entre modelos
│   ├── middlewares/
│   │   ├── authMiddleware.js # Middleware de verificación JWT
│   │   ├── validateCategoria.js # Validaciones para Categorías
│   │   ├── validateProducto.js # Validaciones para Productos
│   │   └── validateVenta.js   # Validaciones para Ventas
│   ├── controllers/
│   │   ├── authController.js # Controlador para emisión de tokens JWT
│   │   ├── categoriaController.js
│   │   ├── productoController.js
│   │   └── ventaController.js
│   ├── routes/
│   │   ├── authRoutes.js    # Rutas de autenticación
│   │   ├── categoriaRoutes.js
│   │   ├── productoRoutes.js
│   │   └── ventaRoutes.js
│   └── app.js              # Punto de entrada y servidor Express
├── .env                   # Variables de entorno
└── package.json           # Configuración de dependencias
```

Instrucciones Paso a Paso

PASO 1: Inicialización e Instalación

1. Inicializa el proyecto con `npm init -y` dentro de la carpeta `api_inventario-ventas/`.
2. Agrega "type": "module" en el archivo `package.json` para habilitar la sintaxis de importación de ES Modules.
3. Instala los paquetes necesarios: `express`, `sequelize`, `pg`, `pg-hstore`, `express-validator`, `dotenv` y `jsonwebtoken`.

PASO 2: Configuración de Base de Datos y Conexión

1. Crea la base de datos llamada `inventario_tienda_db` en PostgreSQL.
2. Crea el archivo `.env` configurando las variables de entorno necesarias:
`PORT=3000`
`DATABASE_URL=postgres://usuario:contraseña@localhost:5432/inventario_tienda_db`
`JWT_SECRET=secreto_inventario_jwt_2026`
3. En `src/config/db.js`, crea y exporta la instancia de Sequelize configurada para el dialecto PostgreSQL.

PASO 3: Definición de los 3 Modelos y sus Relaciones

- Modelo Categoría (`src/models/Categoria.js`):
 - id: Entero, autoincrementable, clave primaria.
 - nombre: Cadena de texto, obligatorio, único (ej. "Laptops", "Periféricos").
 - descripcion: Cadena de texto, opcional.
- Modelo Producto (`src/models/Producto.js`):
 - id: Entero, autoincrementable, clave primaria.
 - sku: Cadena de texto, obligatorio, único (ej. "LAP-DELL-5510").
 - nombre: Cadena de texto, obligatorio.
 - precio: Número decimal, obligatorio (mayor a 0).
 - stock: Entero, obligatorio (mínimo 0).
- Modelo Venta (`src/models/Venta.js`):
 - id: Entero, autoincrementable, clave primaria.
 - cantidad: Entero, obligatorio (mayor a 0).
 - total: Número decimal, obligatorio.
 - cliente: Cadena de texto, obligatorio.
- Establecer Relaciones (`src/models/index.js`):
 - Una Categoría agrupa muchos Productos (`Categoria.hasMany(Producto) / Producto.belongsTo(Categoria)`). Clave foránea: `categoriald`.
 - Un Producto registra muchas Ventas históricas (`Producto.hasMany(Venta) /`

Venta.belongsTo(Producto)). Clave foránea: productold.

- Exporta los tres modelos y la conexión agrupados.

PASO 4: Módulo de Autenticación JWT y Middlewares de Validación

1. Generación de Token (src/controllers/authController.js y src/routes/authRoutes.js): Implementa una ruta de acceso (POST /api/auth/login) que reciba credenciales básicas (ej. email) y genere un token firmado usando jsonwebtoken y JWT_SECRET.
2. Middleware de Protección (src/middlewares/authMiddleware.js): Valida la presencia del encabezado Authorization con esquema Bearer <TOKEN>. Verifica la validez del token mediante jwt.verify() e inyecta los datos decodificados en la petición (req.usuario). Retorna estado 401 Unauthorized o 403 Forbidden si falla.
3. Middlewares de Validación por Modelo (src/middlewares/):
 - Categorías: Validar que nombre no esté vacío.
 - Productos: Validar que sku y nombre no estén vacíos, que precio sea mayor a 0, stock sea entero positivo y exista un categoriaId válido.
 - Ventas: Validar que cantidad y total sean números mayores a 0, que cliente no esté vacío y se envíe un productold válido.
 - En cada middleware, retorna estado 400 Bad Request si no se cumplen las validaciones.

PASO 5: Controladores para las 3 Entidades

1. Implementa funciones CRUD asíncronas (crear, obtenerTodos, obtenerPorId, actualizar, eliminar) para cada uno de los tres modelos.
2. En las consultas de Productos, incluye (include) la información de la Categoría asociada.
3. En las consultas de Ventas, incluye la información del Producto vendido.
4. Responde con estados HTTP estándar (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error).

PASO 6: Enrutamiento Modular y Protección de Rutas

1. Crea las rutas correspondientes en cada archivo (authRoutes.js, categoriaRoutes.js, productoRoutes.js, ventaRoutes.js).
2. Vincula cada verbo HTTP de modificación (POST, PUT, DELETE) con el middleware de autenticación verificarToken y su respectivo validador de datos.

PASO 7: Configuración del Servidor Principal (src/app.js)

1. Inicializa Express, habilita el middleware para JSON (express.json()).
2. Monta los enrutadores bajo sus prefijos: /api/auth, /api/categorias, /api/productos y /api/ventas.

3. Sincroniza la base de datos con Sequelize (sequelize.sync()) e inicia el servidor en el puerto configurado.

PASO 8: Plan de Pruebas y Validación en Postman

Ejecuta tu servidor (node src/app.js) y realiza la secuencia ordenada de pruebas en Postman:

1. Obtención del Token JWT

POST http://localhost:3000/api/auth/login

Body: { "email": "inventario@tienda.com" }

Resultado esperado: Estado 200 OK retornando el token firmado. (Copia este token para los siguientes pasos).

2. Pruebas para el Modelo: Categorías (Ruta Protegida)

POST http://localhost:3000/api/categorias | Headers: Authorization: Bearer <TU_TOKEN>

Body: { "nombre": "Laptops y Portátiles", "descripcion": "Equipos de computación portátil y accesorios" }

Resultado esperado: Estado 201 Created y el objeto registrado.

GET http://localhost:3000/api/categorias

Resultado esperado: Estado 200 OK con el listado de categorías.

3. Pruebas para el Modelo: Productos (Con Relación a Categoría)

POST http://localhost:3000/api/productos | Headers: Authorization: Bearer <TU_TOKEN>

Body: { "sku": "LAP-LEN-01", "nombre": "Lenovo ThinkPad E14", "precio": 750.00, "stock": 15, "categoricalId": 1 }

Resultado esperado: Estado 201 Created vinculando el producto a la categoría con ID 1.

GET http://localhost:3000/api/productos

Resultado esperado: Estado 200 OK incluyendo la categoría anidada.

4. Pruebas para el Modelo: Ventas (Con Relación a Producto)

POST http://localhost:3000/api/ventas | Headers: Authorization: Bearer <TU_TOKEN>

Body: { "cantidad": 2, "total": 1500.00, "cliente": "Empresa Tech Solutions C.A.", "productid": 1 }

Resultado esperado: Estado 201 Created asociando la venta con el producto ID 1.

GET http://localhost:3000/api/ventas

Resultado esperado: Estado 200 OK mostrando el listado de ventas y los detalles del producto asociado.

PUT http://localhost:3000/api/ventas/1 | Headers: Authorization: Bearer <TU_TOKEN>

Body: { "cantidad": 3, "total": 2250.00, "cliente": "Empresa Tech Solutions C.A. (Actualizado)" }

Resultado esperado: Estado 200 OK confirmando la actualización.

DELETE http://localhost:3000/api/ventas/1 | Headers: Authorization: Bearer <TU_TOKEN>

Resultado esperado: Estado 200 OK con mensaje de eliminación exitosa.